Relatório Projeto Trainee 2025.2 - Estatística

Caio Augusto Marangoni de Mello

Gabriel Cury de Paula

Giovanna Maruyama

Henrique Nardi Farbelow

Samuel Filipe de Almeida Silva

ICMC Júnior

1 Introdução

O presente relatório foi elaborado como parte do **Programa Trainee de Estatística 2025.2**, promovido pela ICMC Júnior – Empresa Júnior do Instituto de Ciências Matemáticas e de Computação da Universidade de São Paulo (USP). O projeto tem como principal objetivo introduzir os trainees aos conceitos fundamentais de Ciência de Dados, proporcionando uma experiência prática e colaborativa na resolução de problemas reais com o uso de ferramentas estatísticas e computacionais.

Ao longo do projeto, foi utilizada a base de dados **HR Analytics**, que contém informações relacionadas a colaboradores de uma empresa fictícia, incluindo variáveis como idade, cargo, tempo de empresa, nível de satisfação, salário, desempenho, entre outras. Essa base é amplamente utilizada em estudos de Recursos Humanos e serve como ponto de partida para análises voltadas à compreensão do perfil dos funcionários e dos fatores que influenciam seu comportamento dentro da organização.

O **objetivo geral** deste trabalho é explorar, tratar e analisar a base de dados, aplicando técnicas de *Data Science* e *Machine Learning* para identificar padrões e extrair insights relevantes. As etapas envolvem desde a preparação dos dados até a construção de modelos preditivos, sempre com foco em desenvolver habilidades técnicas e analíticas.

Como **objetivo específico**, o grupo escolheu desenvolver um modelo capaz de **prever se um funcionário deixará a empresa (attrition)**, a partir de suas características demográficas, profissionais e de desempenho. Essa análise busca compreender quais fatores mais contribuem para a rotatividade de funcionários, permitindo identificar perfis de colaboradores com maior propensão a sair e auxiliando na criação de estratégias de retenção mais eficientes.

Assim, este relatório apresenta as etapas seguidas durante o desenvolvimento do projeto, bem como os resultados obtidos e as interpretações derivadas das análises realizadas.

2 Etapa 1 - Preparação da Base de Dados

A etapa de preparação da base de dados teve como principal objetivo garantir que o conjunto estivesse limpo, padronizado e pronto para as análises estatísticas e de *machine learning*. Todo o processamento foi realizado em ambiente *Google Colab*, utilizando as bibliotecas `pandas`, `numpy` e `matplotlib`.

2.1 Importação e Inspeção Inicial

A base **HR Analytics** foi importada e inspecionada para compreender sua estrutura e identificar possíveis problemas. Foram verificadas a quantidade de linhas e colunas, os tipos de dados de cada variável e a presença de valores ausentes.

```
import pandas as pd
df = pd.read_csv('HR_Analytics.csv')
df.info()
df.head()
```

Essa análise inicial permitiu identificar variáveis categóricas, numéricas e inconscistências como dados faltantes (NaN) em algumas colunas. Além disso, com o objetivo de facilitar o entendimento e a interpretação, adicionamos legendas para todas as variavéis.

2.2 Análise das Colunas

Com a listagem de variáveis (`df.columns`) e a inspeção estrutural (`df.info()`), avaliamos tipo de dado, variabilidade , valores ausentes (`isnull().sum()`) e utilidade preditiva de cada coluna, sempre com foco no objetivo de prever *attrition*.

2.2.1 Valores ausentes por coluna:

Primeiro, foi feita a contagem de ausências para mapear o tratamento de cada variável:

```
contagem_nan = df.isnull().sum()
```

2.2.2 Colunas sem valor preditivo prático:

Foram identificadas variáveis com valor constante ou de caráter meramente identificador: `StandardHours`, `Over18`, `EmployeeCount` e `EmployeeNumber`. A análise de `unique()`

e `value_counts()` mostrou, por exemplo, que `StandardHours` é constante (80 em todas as 1470 linhas). Essas colunas foram removidas por não contribuírem para o aprendizado do modelo. Assim removemos essas colunas:

```
df.drop(columns=['StandardHours','Over18','EmployeeCount','EmployeeNumber'],
inplace=True)
```

2.2.3 Relação entre `HourlyRate`, `DailyRate` e `MonthlyRate`:

Levantamos a hipótese de que as três taxas fossem fortemente proporcionais (e, portanto, redundantes). Após verificar correlações entre elas, observou-se correlação baixa, contrariando a hipótese inicial; por isso, as três foram mantidas para as próximas etapas.

2.3 Padronização dos Dados

Após a análise das colunas e a remoção das variáveis sem valor preditivo prático, a próxima etapa consistiu em padronizar a base de dados, tratando os valores ausentes e garantindo a consistência das informações. Essa etapa foi essencial para preparar o conjunto de dados para as análises estatísticas e a construção do modelo de *machine learning*.

2.3.1 Decisão de imputação por moda:

Para variáveis categóricas com número reduzido de ausências, a imputação foi feita utilizando o valor mais frequente (moda). Por exemplo, na coluna `BusinessTravel`, foram encontrados cerca de 70 valores ausentes em um total de 1470 registros. Como a maior parte dos colaboradores já pertencia à categoria `Travel_Rarely` (quase 1000 ocorrências), decidiu-se substituir os valores faltantes por essa moda. Essa escolha visa manter a proporção natural da variável e evitar distorções artificiais na distribuição das categorias.

```
df['BusinessTravel'].fillna(df['BusinessTravel'].mode()[0], inplace=True)
```

De modo semelhante, outras colunas categóricas com pequenas quantidades de valores ausentes foram preenchidas usando o mesmo princípio, sempre priorizando a categoria predominante na amostra.

2.3.2 Imputação proporcional:

Para preencher a coluna `Gender`, optou-se por uma abordagem mais cuidadosa. Como essa variável é relevante em estudos relacionados à retenção de funcionários e poderia influenciar diretamente os resultados do modelo, decidiu-se manter a proporção original entre homens e mulheres. A utilização da moda inflaria um dos grupos, gerando viés.

O código fez a contagem de homens e mulheres, calculou o vetor de probabilidades e preencheu os valores ausentes segundo essas probabilidades. Foi utilizada a função `np.random.seed(42)` para garantir a reprodutibilidade dos resultados.

Seguindo exatamente a mesma ideia, preenchemos a coluna 'OverTime' da mesma forma, pois acreditamos que a proporção também seja importante nesse caso.

2.3.3 Imputação em variáveis numéricas:

Para as variáveis numéricas com valores faltantes, como `MonthlyIncome`, optou-se pela imputação pela média. Essa decisão foi tomada porque a distribuição dessa variável não apresentava forte assimetria, e o número de valores ausentes era reduzido, o que torna a média uma boa estimativa central.

```
df['MonthlyIncome'].fillna(df['MonthlyIncome'].mean(), inplace=True)
```

A utilização da média preserva a tendência geral dos dados sem alterar significativamente a dispersão, garantindo que o comportamento estatístico da variável permaneça coerente.

2.3.4 Verificação pós-imputação:

Após o preenchimento dos valores ausentes, uma nova verificação foi realizada para confirmar que não restavam `NaN` na base:

```
df.isnull().sum()
```

O resultado retornou zero valores ausentes em todas as colunas, indicando que a base está completa e pronta para as análises subsequentes.

Ao final dessa etapa, a base de dados estava devidamente tratada, padronizada e livre de inconsistências, servindo como ponto de partida para as próximas fases do projeto: a visualização dos dados, os testes de correlação e a modelagem preditiva do *attrition*.

3 Etapa 2 - Visualização e Exploração dos Dados

Com a base devidamente tratada e padronizada, iniciou-se a etapa de **Visualização e Exploração dos Dados**, cujo objetivo é compreender o comportamento individual e conjunto das variáveis. Essa etapa foi dividida em duas partes: uma análise univariada e, posteriormente, análises que relacionam o *attrition* com outras variáveis relevantes.

3.1 Análise Univariada

A análise univariada busca observar a distribuição e dispersão de cada variável de forma isolada, possibilitando a identificação de padrões, tendências e possíveis outliers.

3.1.1 Variáveis Categóricas

Se tratando de variáveis categóricas, o foco está em mostrar a contagem e a proporção (ou popularidade) de cada categoria presente na variável. Para isso, um simples gráfico de barras de contagem é suficiente para evidenciar os padrões de interesse.

Por exemplo, a variável `JobSatisfaction` apresentou predominância nas categorias 3 e 4 (em uma escala de 1 a 4), indicando alto nível de satisfação dos colaboradores com suas funções. Da mesma forma, a variável `Department` mostrou que a maior parte dos funcionários pertence ao setor de `Research & Development`, seguido por `Sales`.

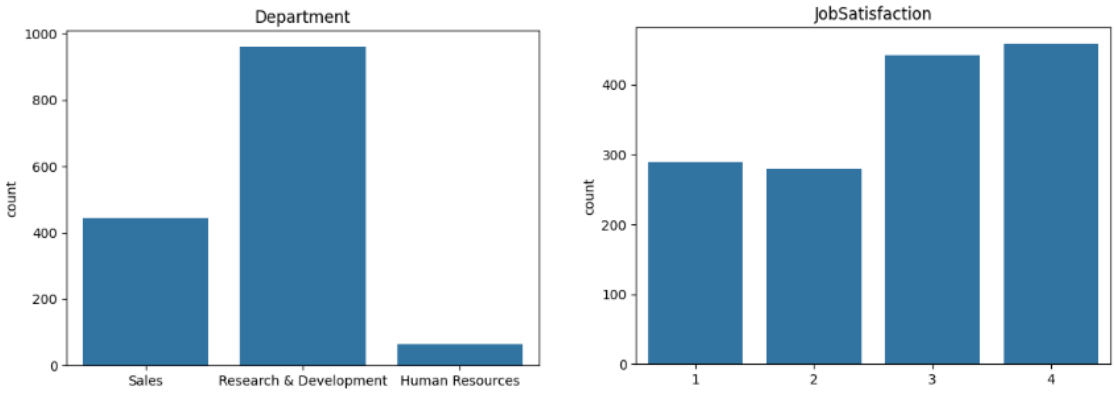

Figura 1: Painel complementar relacionado a *Department* e a *JobSatisfaction*.

Além disso, a variável `Attrition` (variável-alvo do estudo) mostrou-se desbalanceada: aproximadamente 16% dos colaboradores deixaram a empresa, enquanto 84% permanecem. Esse desequilíbrio é comum em dados reais de recursos humanos e será considerado nas etapas de modelagem.

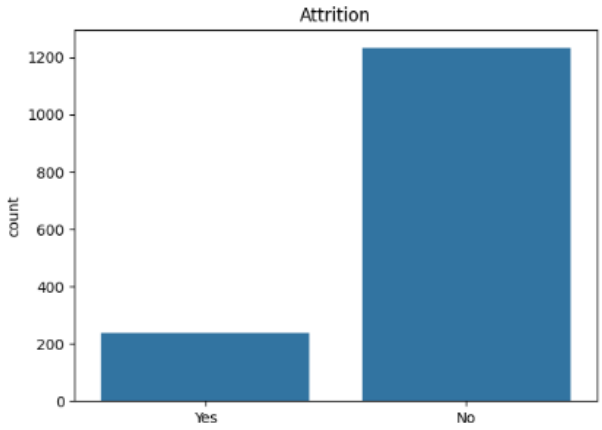

Figura 2: Painel complementar relacionado à variável *Attrition*.

3.1.2 Variáveis Numéricas

Para as variáveis numéricas, o objetivo foi observar a distribuição e a presença de valores extremos. Foram utilizados histogramas e boxplots, que permitem visualizar tanto a forma da distribuição quanto a dispersão dos dados.

Por exemplo, ao analisar a variável `MonthlyIncome`, observou-se uma distribuição assimétrica à direita, com a maioria dos colaboradores recebendo entre 3.000 e 8.000, e alguns casos isolados acima de 20.000. Tal assimetria é reforçada pelo boxplot.

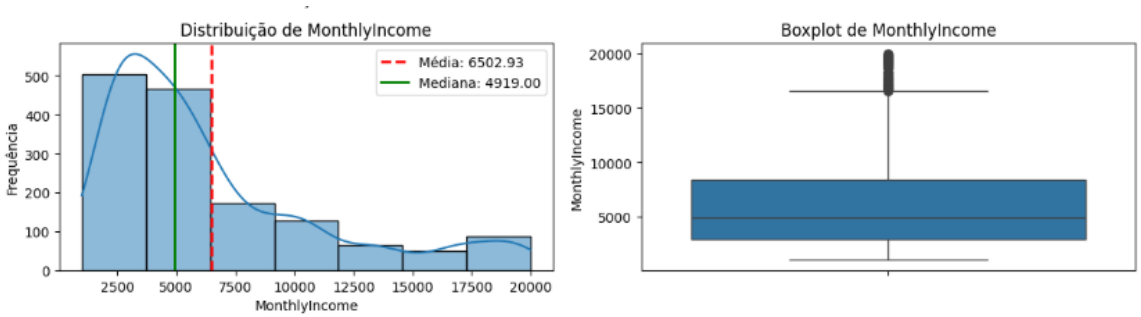

Figura 3: Painel complementar relacionado à variável *MonthlyIncome*.

De modo geral, as variáveis numéricas apresentaram distribuições compatíveis com o esperado em um ambiente empresarial.

3.2 Análise Bivariada

Após a análise univariada, foi realizada a **análise bivariada**, com o objetivo de investigar como as variáveis independentes se relacionam com o *attrition*. Essa etapa permite identificar padrões de comportamento e possíveis fatores de influência na decisão dos colaboradores de permanecer ou sair da empresa.

3.2.1 Variáveis Categóricas

Se tratando de variáveis categóricas, o foco esteve em compreender como a proporção de desligamentos varia entre diferentes grupos. Por exemplo, na variável `BusinessTravel`, observou-se uma taxa de *attrition* mais alta entre colaboradores que viajam frequentemente (`Travel_Frequently`) em comparação àqueles que viajam raramente (`Travel_Rarely`). Esse padrão sugere que viagens constantes podem impactar a permanência do funcionário.

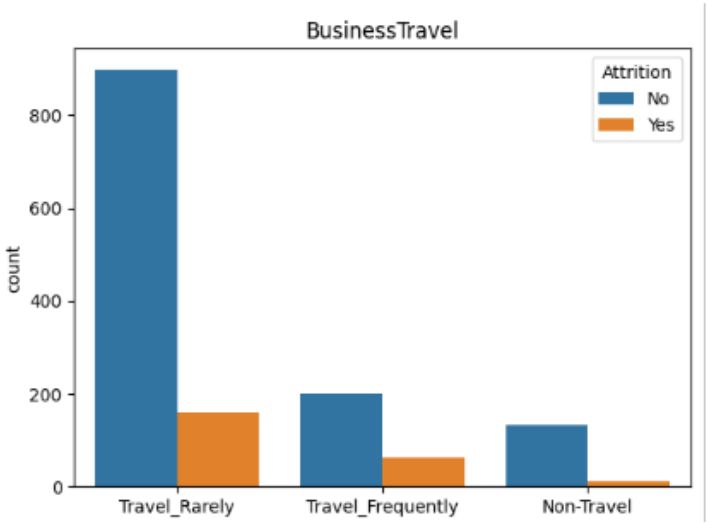

Figura 4: Relação à variável *BusinessTravel*.

Outro caso relevante foi a variável `MaritalStatus`. Trabalhadores solteiros apresentaram proporções mais elevadas de desligamento, indicando que o estado civíl pode estar diretamente relacionado ao risco de rotatividade.

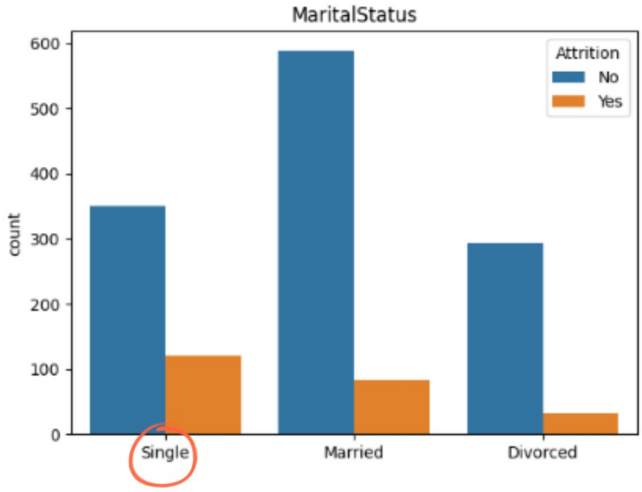

Figura 5: Relação à variável *MaritalStatus*.

3.2.2 Variáveis Numéricas

Para as variáveis numéricas, o objetivo foi avaliar diferenças nas distribuições entre funcionários que permaneceram na empresa e os que saíram. O boxplot da variável `Age` mostrou que o *attrition* é mais comum entre colaboradores mais jovens, enquanto os funcionários mais experientes tendem a permanecer.

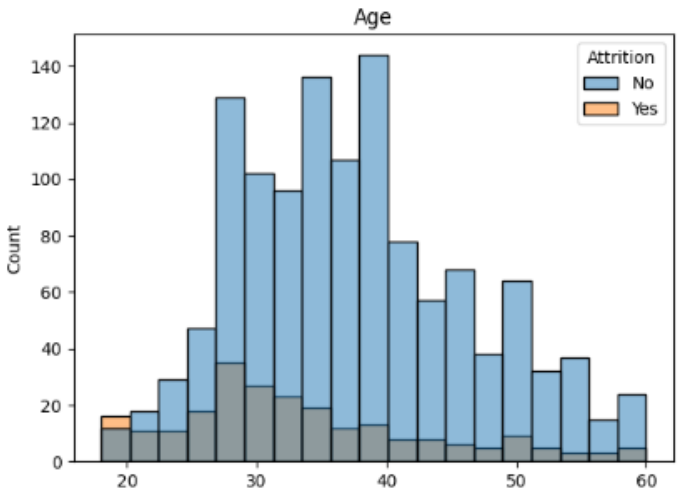

Figura 6: Relação à variável *Age*.

De modo semelhante, observou-se que o salário mensal (`MonthlyIncome`) é, em média, menor entre os funcionários que deixaram a empresa, o que sugere uma correlação entre menor remuneração e maior propensão ao desligamento.

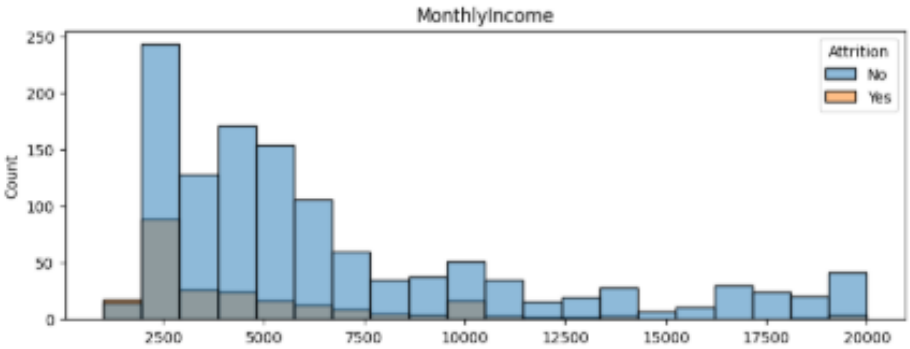

Figura 7: Relação à variável *MonthlyIncome*.

Por fim, analisando o tempo de empresa (`YearsAtCompany`), constatou-se que colaboradores com menos tempo de vínculo apresentaram maior índice de saída, o que reforça a hipótese de que a rotatividade é mais alta nos primeiros anos de trabalho.

4 Etapa 3 - Testes de Correlação

Após a análise bivariada, foi realizada a etapa de **testes de correlação** com o objetivo de avaliar a relação entre as variáveis numéricas da base de dados. Essa análise permite identificar atributos que possuem associação linear significativa, tanto entre si quanto com a variável-alvo `Attrition`. A compreensão dessas relações é fundamental para reduzir redundâncias e orientar a seleção de variáveis mais relevantes para a modelagem.

4.1 Cálculo e Visualização da Matriz de Correlação

Foi calculada a matriz de correlação de Pearson, adequada para variáveis contínuas, uma vez que coeficiente de Pearson mede a intensidade e a direção da relação linear entre duas variáveis, variando entre -1 (correlação negativa perfeita) e +1 (correlação positiva perfeita). Valores próximos de zero indicam ausência de relação linear significativa.

No gráfico, tons avermelhados indicam correlação positiva e tons azulados indicam correlação negativa.

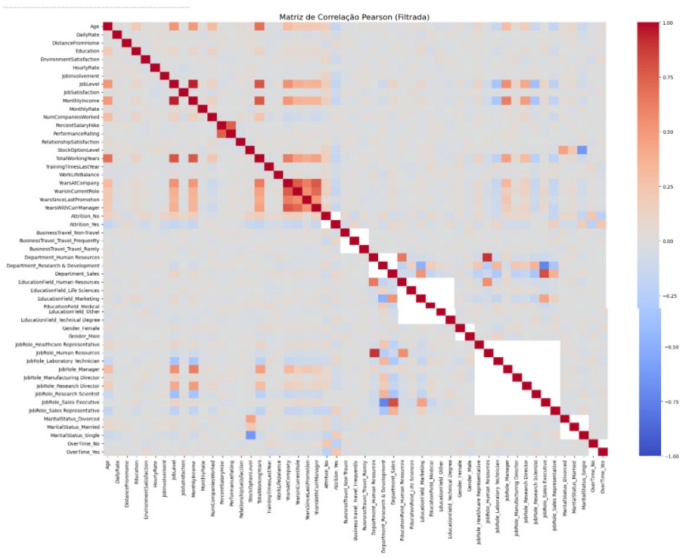

Figura 8: Matriz de correlação entre variáveis numéricas (Pearson)

Como a base também contém variáveis ordinais, especialmente as relacionadas à satisfação (`JobSatisfaction`, `EnvironmentSatisfaction`, `RelationshipSatisfaction`), foi aplicado o coeficiente de **Spearman**. Diferentemente de Pearson, Spearman avalia relações *monotônicas*, não exigindo linearidade, sendo mais adequado para esse tipo de dado.

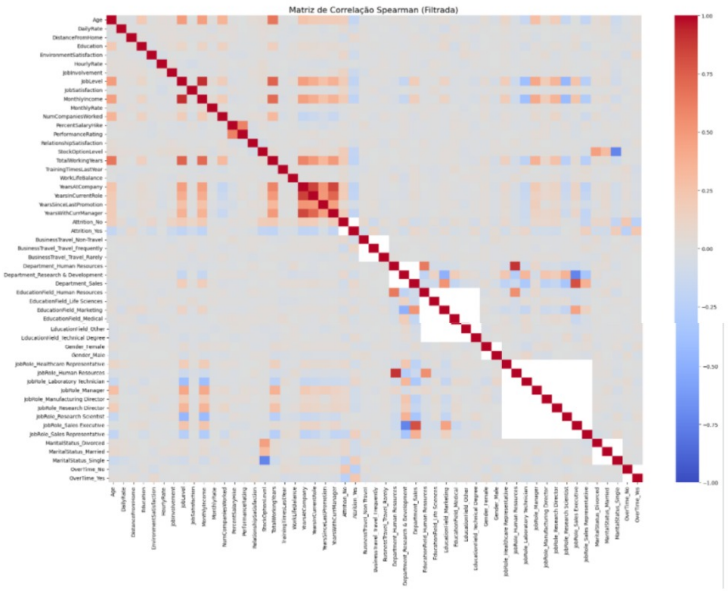

Figura 9: Matriz de correlação entre variáveis numéricas (Spearman).

4.2 Análise das Correlações Observadas

Após a obtenção das duas matrizes de correlação, foi feita uma análise comparativa entre os resultados de Pearson e Spearman, buscando identificar os pares de variáveis que apresentaram correlações fortes em ambos os métodos. Essa verificação tem como objetivo validar relações consistentes, tanto lineares quanto monotônicas, garantindo maior robustez às conclusões. A Tabela 1 apresenta o ranking dos pares mais correlacionados, presentes simultaneamente no Top 10 de ambos os métodos.

Tabela 1: Pares comuns entre as correlações de Pearson e Spearman.

#	**Variável 1**	**Variável 2**
1	Department_Sales	JobRole_Sales Executive
2	JobLevel	MonthlyIncome
3	YearsAtCompany	YearsWithCurrManager
4	YearsInCurrentRole	YearsWithCurrManager
5	Department_Research & Development	JobRole_Sales Executive
6	Department_Human Resources	JobRole_Human Resources
7	YearsAtCompany	YearsInCurrentRole
8	JobLevel	TotalWorkingYears
9	MonthlyIncome	TotalWorkingYears

Esses resultados indicam que há forte coerência entre os métodos de Pearson e Spearman, evidenciando que as relações identificadas não dependem apenas da linearidade, mas também de tendências monotônicas consistentes. Destaque especial deve ser dado às variáveis relacionadas à `JobLevel`, `MonthlyIncome` e `TotalWorkingYears`, que aparecem em múltiplos pares e representam fatores estruturais diretamente associados à senioridade e estabilidade do colaborador dentro da empresa.

5 Etapa 4 - Previsão com Machine Learning

Com as etapas de exploração e correlação concluídas, iniciou-se o processo de **modelagem preditiva**, cujo objetivo é construir um modelo capaz de prever o desligamento de colaboradores (`Attrition`).

5.1 Escolha de Modelos

A etapa de **escolha de modelos** teve início com a preparação dos dados para o processo de modelagem. Antes da definição do algoritmo de aprendizado, foi necessário estruturar o conjunto de dados em variáveis independentes (`X`) e variável-alvo (`y`), além de identificar os tipos de colunas e realizar a divisão entre os conjuntos de treino e teste.

5.1.1 Definição de X e y

A variável `Attrition`, que indica se o colaborador deixou ou não a empresa, foi mapeada para valores numéricos binários — 1 representando "Yes" (funcionário saiu) e 0 representando "No" (funcionário permaneceu). Em seguida, definiu-se `X` como a matriz de covariáveis (todas as demais colunas do conjunto de dados) e `y` como a variável-alvo.

Assim, possibilitou-se o uso de algoritmos supervisionados de classificação, que exigem variáveis numéricas para o treinamento.

5.1.2 Identificação de colunas numéricas e categóricas

Com o objetivo de aplicar os pré-processamentos adequados a cada tipo de variável, as colunas foram separadas em dois grupos: `numeric_features` (colunas numéricas) e `categorical_features` (colunas categóricas). A identificação foi feita com base nos tipos de dados (`int64`, `float64` e `object`).

Assim, garantiu-se maior controle no tratamento das variáveis, permitindo aplicar normalização, codificação ou imputações de forma direcionada em cada grupo.

5.1.3 Divisão dos dados

Por fim, os dados foram divididos em conjuntos de treino e teste, na proporção de 75% para o treinamento e 25% para o teste, garantindo que o modelo fosse avaliado em dados não vistos durante o aprendizado. O parâmetro `random_state=33` foi definido para garantir a reprodutibilidade dos resultados.

A divisão dos dados representa a etapa final da preparação para o treinamento, assegurando que o conjunto de teste sirva exclusivamente para avaliar o desempenho do modelo escolhido nas próximas fases.

5.2 Pré-processamento e Pipeline

Com os dados devidamente divididos entre treino e teste, foi realizada a etapa de **pré-processamento**, responsável por preparar as variáveis para o treinamento do modelo. Essa fase é essencial para garantir que todas as colunas estejam em formato numérico e padronizado, permitindo que o algoritmo de aprendizado de máquina processe corretamente as informações.

No conjunto `HR Analytics`, há variáveis de naturezas distintas — numéricas contínuas, discretas e categóricas. Por esse motivo, adotou-se um fluxo de tratamento separado para cada tipo de variável, unificado posteriormente por meio de um *pipeline*.

O pré-processamento foi estruturado da seguinte forma:

- **Variáveis numéricas:** passaram por padronização utilizando o `StandardScaler`, que centraliza os dados em torno da média e os reescala para desvio padrão igual a 1, para evitar que variáveis com diferentes magnitudes influenciem desproporcionalmente o modelo.

- **Variáveis categóricas:** foram convertidas em formato numérico por meio do `OneHotEncoder`, que cria variáveis binárias (0 e 1) para cada categoria existente, o processo garante que o modelo interprete corretamente as categorias sem atribuir ordens artificiais a elas.

  Em seguida, as duas transformações foram combinadas com o uso do `ColumnTransformer`, que aplica cada tipo de transformação ao grupo de colunas correspondente.

Por fim, foi criado um `Pipeline` integrando o pré-processamento e o modelo de aprendizado, garantindo que todas as etapas (transformações e treinamento) fossem executadas em sequência, com o objetivo de evitar vazamento de dados entre as etapas de treino e teste e facilitar a reprodutibilidade do processo.

A utilização do `Pipeline` garante que o fluxo completo — desde o tratamento dos dados até a predição — seja automatizado e reproduzível.

Dessa forma, ao final do pré-processamento, todos os dados categóricos foram codificados, as variáveis numéricas padronizadas e o conjunto final ficou pronto para o treinamento do modelo de predição de *attrition*.

.

5.3 Treinamento e Avaliação do Modelo

Com o pipeline de pré-processamento definido, diferentes algoritmos de classificação supervisionada foram testados com o objetivo de identificar qual apresentaria melhor desempenho na previsão de *attrition*. Os modelos avaliados foram: **Regressão Logística**, **Random Forest**, **Gradient Boosting Machine (GBM)** e **XGBoost**. Todos os experimentos foram realizados dentro da mesma estrutura de pipeline, garantindo padronização no pré-processamento e na avaliação.

5.3.1 Regressão Logística

A **Regressão Logística** foi o primeiro modelo testado no projeto, servindo como *baseline* para comparação com os demais algoritmos. Seu objetivo é estimar a probabilidade de um colaborador deixar a empresa (`Attrition = 1`) a partir de uma combinação linear das variáveis explicativas, sendo um modelo simples.

Treinamento e Avaliação: O modelo foi implementado dentro de um `Pipeline`, garantindo o mesmo pré-processamento aplicado às demais abordagens. O treinamento foi realizado com o conjunto de treino e a avaliação feita sobre o conjunto de teste, utilizando as métricas de acurácia, precisão, recall e F1-Score.

Tabela 2: Relatório de Classificação (Precision, Recall, F1-Score) da Regressão Logística.

Classe	**Precision**	**Recall**	**F1-Score**	**Support**
Não Saiu (0)	0.95	0.73	0.82	306
Saiu (1)	0.37	0.81	0.51	62
Accuracy			**0.74**	368
Macro Avg	0.66	0.77	0.67	368
Weighted Avg	0.85	0.74	0.77	368

Resultados e Análise: A Regressão Logística obteve acurácia de 0.74. O modelo mostrou alta sensibilidade para a classe "Saiu (1)" (recall = 0.81), ou seja, captura a maior parte dos casos de desligamento, mas com baixa precisão nessa classe (0.37), indicando muitos falsos positivos (prevê "saiu" quando na verdade não saiu). Para a classe majoritária "Não Saiu (0)", a precisão é alta (0.95) — quando prevê "não saiu", quase sempre acerta —, porém o recall é 0.73, o que revela perdas dessa classe em função do viés do modelo para marcar mais positivos.

Esse comportamento é esperado em modelos lineares aplicados a bases desbalanceadas, nas quais a proporção de funcionários que permanecem é muito maior do que a de desligamentos.

5.3.2 Random Forest

O modelo `RandomForestClassifier` foi o primeiro a ser treinado e serviu como referência inicial.

Treinamento do modelo: O treinamento foi realizado utilizando o conjunto de treino (`X_train` e `y_train`) por meio do método `fit()`, que ajusta internamente o pré-processamento e o classificador do pipeline.

Predições e avaliação: Após o treinamento, foram realizadas as predições no conjunto de teste para avaliar o desempenho do modelo. Para avaliar a qualidade das previsões, foram utilizadas as principais métricas de classificação binária: **Acurácia**, **Precisão**, **Recall** e **F1-Score**. Essas métricas foram obtidas a partir da biblioteca `scikit-learn`.

A métrica de **acurácia** indica a proporção total de previsões corretas, enquanto a **precisão** mede a capacidade do modelo de não classificar incorretamente funcionários que não saíram como se tivessem saído. O **recall** avalia a capacidade de identificar corretamente os colaboradores que efetivamente deixaram a empresa, e o **F1-score** combina precisão e recall em uma única métrica balanceada.

Matriz de confusão (Seaborn): Para melhor visualização dos acertos e erros de classificação, foi plotada a matriz de confusão. Ela mostra a quantidade de verdadeiros positivos (colaboradores corretamente identificados como desligados), verdadeiros negativos (funcionários corretamente classificados como permanecentes) e as classificações incorretas.

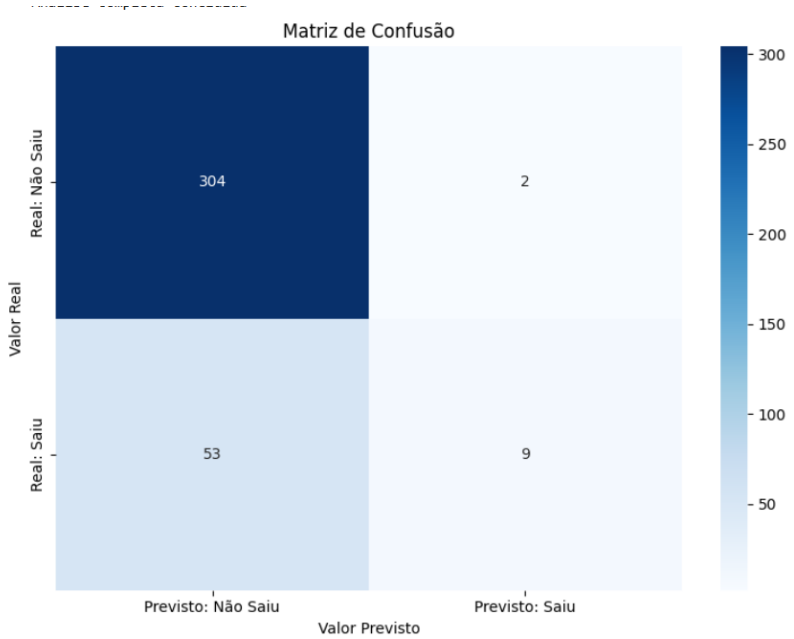

Figura 10: Matriz de confusão para o modelo Random Forest.

Resultados e análise: Após o treinamento, foi calculado o **score de avaliação** do modelo no conjunto de teste, utilizando o método `score()` do pipeline. Esse valor representa a acurácia geral do modelo — ou seja, a proporção de previsões corretas.

O modelo **Random Forest** apresentou desempenho satisfatório, com **acurácia de aproximadamente 0.85**, evidenciando boa capacidade de generalização e distinção entre os casos de *attrition* e *non-attrition*.

Além da acurácia geral, foi gerado o **Relatório de Classificação**, que resume as principais métricas de desempenho — *Precision*, *Recall* e *F1-Score* — para cada classe.

Tabela 3: Relatório de Classificação (Precision, Recall, F1-Score) do modelo Random Forest.

Classe	**Precision**	**Recall**	**F1-Score**	**Support**
Não Saiu (0)	0.85	0.99	0.92	306
Saiu (1)	0.82	0.15	0.25	62
Accuracy			**0.85**	368
Macro Avg	0.83	0.57	0.58	368
Weighted Avg	0.85	0.85	0.80	368

A análise do relatório mostra que o modelo obteve excelente desempenho na identificação dos funcionários que **não saíram da empresa** (classe 0), atingindo *recall* de 0.99 e *f1-score* de 0.92. Por outro lado, o desempenho na detecção dos funcionários que **saíram da empresa** (classe 1) foi inferior, com *recall* de 0.15 e *f1-score* de 0.25. Esse comportamento é esperado em bases desbalanceadas, nas quais a maioria das observações pertence a uma das classes.

Apesar dessa limitação, o modelo apresentou uma **acurácia global de 85%**, com boa média ponderada (*weighted avg*) de 0.80 para o F1-score, indicando que, de forma geral, as previsões foram consistentes. Ainda assim, o baixo *recall* da classe minoritária sugere que técnicas de balanceamento ou ajuste de limiar de decisão poderiam ser exploradas em versões futuras do modelo.

Além disso, foi analisada a **importância das variáveis** no modelo, que indica o quanto cada atributo contribui para a decisão de classificação. Essa análise foi feita a partir do atributo `feature_importances_` do modelo, e visualizada em um gráfico de barras com as dez variáveis mais relevantes.

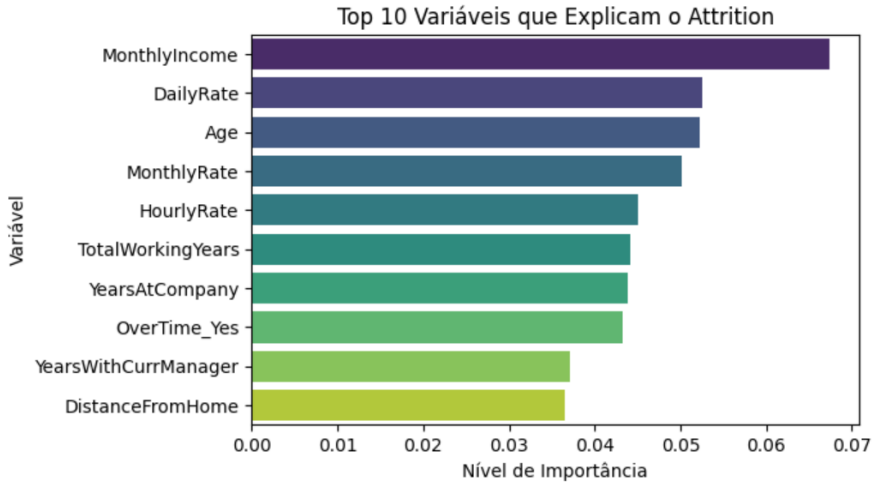

Figura 11: Top 10 variáveis que mais influenciam o *attrition* no modelo Random Forest.

As variáveis com maior relevância incluem principalmente fatores relacionados à estrutura e experiência profissional, como `MonthlyIncome`, `DailyRate`, `Age` e `MonthlyRate`. Esses resultados sugerem que colaboradores com rendas mensais maiores e maior tempo de empresa têm menor probabilidade de deixar a empresa, o que está de acordo com as análises exploratórias e correlações realizadas anteriormente.

5.3.3 Gradient Boosting Machine (GBM)

O modelo de **Gradient Boosting Machine (GBM)** treina múltiplas árvores de decisão de forma sequencial, onde cada nova árvore tenta corrigir os erros cometidos pelas anteriores. Esse processo iterativo permite capturar relações complexas entre as variáveis, mas também exige maior cuidado com parâmetros para evitar *overfitting*.

Treinamento do modelo: Assim como nas abordagens anteriores, o modelo foi inserido em um `Pipeline` que engloba o pré-processamento (padronização e codificação) e o classificador. O treinamento foi feito com o conjunto de treino, utilizando o algoritmo `GradientBoostingClassifier` da biblioteca `scikit-learn`.

Matriz de confusão (Seaborn): Para a visualização dos acertos e erros de classificação, também foi plotada a matriz de confusão, indicando a quantidade de verdadeiros positivos (colaboradores corretamente identificados como desligados), verdadeiros negativos (funcionários corretamente classificados como permanecentes) e as classificações incorretas.

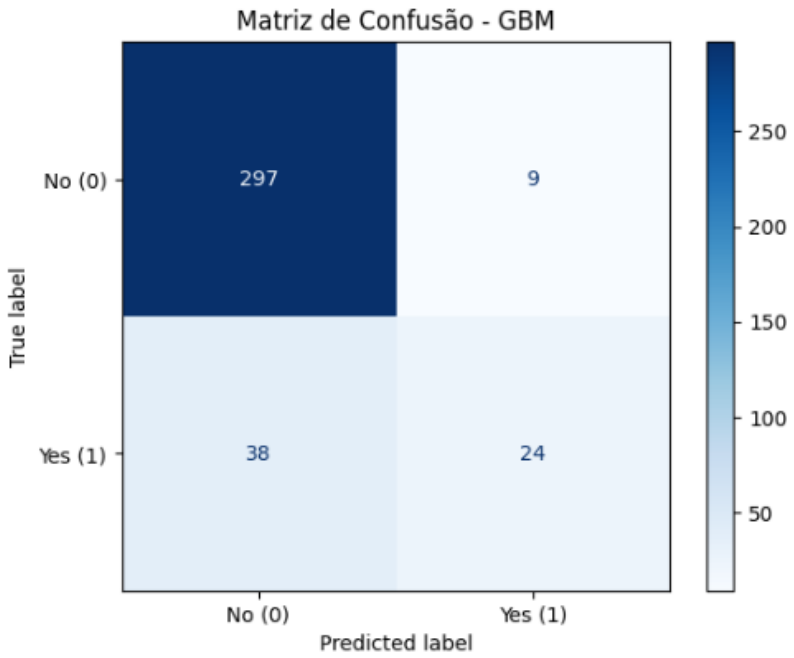

Figura 12: Matriz de Congusão do modelo GBM

Avaliação do modelo: Após o treinamento, as predições foram geradas sobre o conjunto de teste, e as métricas de desempenho calculadas. As principais métricas — *Accuracy*, *Precision*, *Recall* e *F1-Score*.

Tabela 4: Relatório de Classificação (Precision, Recall, F1-Score) do modelo GBM.

Classe	**Precision**	**Recall**	**F1-Score**	**Support**
Não Saiu (0)	0.89	0.97	0.93	306
Saiu (1)	0.73	0.39	0.51	62
Accuracy			**0.87**	368
Macro Avg	0.81	0.68	0.72	368
Weighted Avg	0.86	0.87	0.86	368

Resultados e análise: O modelo **Gradient Boosting Machine (GBM)** apresentou desempenho consistente, com **acurácia geral de 0.87**. A classe majoritária ("Não Saiu") manteve excelente desempenho, com *recall* de 0.97 e *f1-score* de 0.93, indicando que o modelo continua muito eficaz em identificar corretamente os colaboradores que permanecem na empresa.

Além disso, observou-se um aumento considerável no **recall da classe minoritária ("Saiu")**, que passou de 0.17 para **0.39**, acompanhado de um *f1-score* de 0.51, ou seja, o modelo passou a reconhecer de forma mais eficiente os funcionários que realmente deixaram a empresa, ainda que à custa de uma leve redução na precisão geral.

O resultado reflete o funcionamento do algoritmo de **Gradient Boosting**, uma vez que, ele conseguiu capturar padrões mais sutis entre as variáveis. Mesmo sem balanceamento explícito de classes, o modelo demonstrou maior sensibilidade à classe minoritária.

5.3.4 XGBoost

Como alternativa ao problema de desbalanceamento observado no modelo GBM, foi implementado o **XGBoost** (*Extreme Gradient Boosting*), uma versão otimizada e de alta performance do algoritmo de *Gradient Boosting*, ele é reconhecido por sua eficiência e capacidade de lidar com grandes volumes de dados tabulares.

Diferentemente do GBM tradicional, o XGBoost permite ajustar diretamente o peso das classes por meio do parâmetro `scale_pos_weight`. Esse parâmetro foi configurado com base na razão entre as classes majoritária ("Não Saiu") e minoritária ("Saiu"), de forma a aumentar o custo dos erros cometidos ao classificar falsamente os colaboradores que realmente deixaram a empresa.

Treinamento e avaliação: O modelo foi treinado com o conjunto de treino e posteriormente avaliado no conjunto de teste. As previsões (`predict`) e as probabilidades de saída (`predict_proba`) foram utilizadas para calcular as principais métricas.

Matriz de confusão (Seaborn): Igualmente aos outros casos, foi desenvolvida a matriz de confusão para identificar a quantidade de verdadeiros positivos (colaboradores corretamente identificados como desligados), verdadeiros negativos (funcionários corretamente classificados como permanecentes) e as classificações errôneas.

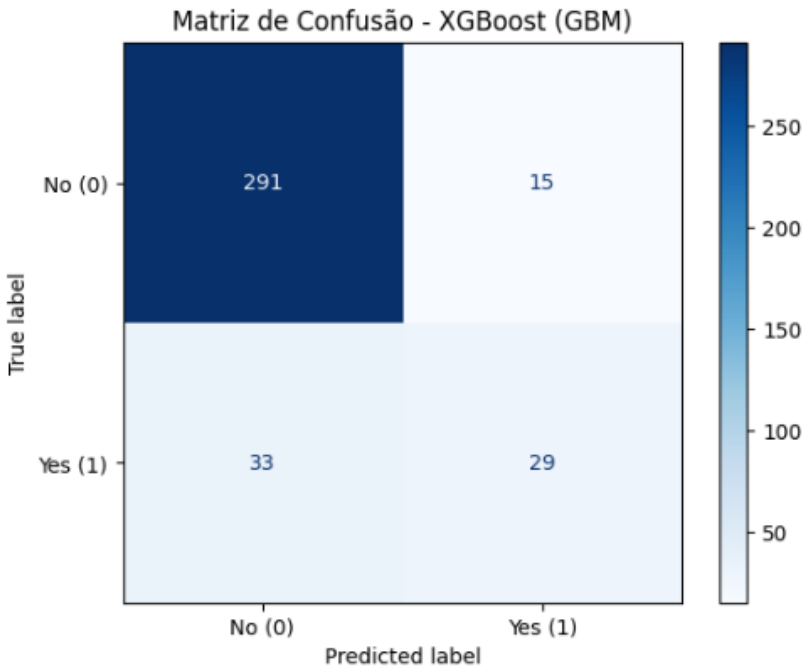

Figura 13: Matriz de Congusão do modelo XGBoost

Observação :A matriz de confusão mostra que o XGBoost identificou corretamente 29 dos 62 casos de *attrition*, enquanto o GBM anterior reconhecia apenas 24. Além disso, a quantidade de falsos positivos (funcionários previstos como "Saiu" mas que permaneceram) permaneceu controlada, demonstrando bom equilíbrio entre *precision* e *recall*.

Avaliação do modelo: Assim como nos outros modelos, após o treinamento, as predições foram geradas sobre o conjunto de teste, e as métricas de desempenho calculadas — *Accuracy*, *Precision*, *Recall* e *F1-Score*.

Tabela 5: Relatório de Classificação (Precision, Recall, F1-Score) do modelo XGBoost.

Classe	**Precision**	**Recall**	**F1-Score**	**Support**
Não Saiu (0)	0.90	0.95	0.92	306
Saiu (1)	0.66	0.47	0.55	62
Accuracy			**0.87**	368
Macro Avg	0.78	0.71	0.74	368
Weighted Avg	0.86	0.87	0.86	368

Resultados e análise: O modelo **XGBoost** apresentou desempenho superior em relação às abordagens anteriores, alcançando uma **acurácia geral de 0.87**. A classe majoritária ("Não Saiu") manteve ótimo desempenho, com *recall* de 0.95 e *f1-score* de 0.92, enquanto a classe minoritária ("Saiu") mostrou melhora expressiva, com *recall* de 0.47 e *f1-score* de 0.55.

Essa evolução é resultado direto da utilização do parâmetro `scale_pos_weight`, que ajusta o peso da classe minoritária durante o treinamentO, O ajuste faz com que o algoritmo atribua maior penalidade a erros de predição dos colaboradores que realmente saem da empresa, corrigindo o viés natural em direção à classe majoritária. Como consequência, o modelo tornou-se mais sensível à classe "Saiu", sem comprometer significativamente a precisão geral. Esses resultados mostram que o XGBoost conseguiu corrigir parte da limitação observada nos modelos anteriores, tornando-se mais eficaz em identificar padrões associados ao desligamento de funcionários.Ou seja, a utilização do **balanceamento foi determinante para esse ganho, permitindo ao modelo aprender melhor os sinais que indicam propensão ao** *attrition*.

5.4 Conclusões

Metodologia de Avaliação: F1-Score

Para avaliar e comparar o desempenho dos modelos de *machine learning* desenvolvidos (Regressão Logística, Random Forest, Gradient Boosting e XGBoost), a métrica de performance selecionada como principal foi o **F1-Score**.

A escolha do F1-Score em detrimento de métricas mais simples, como a acurácia, se deve à natureza desbalanceada do conjunto de dados. Em problemas de previsão de rotatividade (*attrition*), é comum que a classe de interesse ("Saiu") seja muito menos frequente do que a classe majoritária ("Não Saiu"). Assim, o F1-Score permite avaliar de forma mais equilibrada o desempenho do modelo em termos de **precisão** (evitar falsos positivos) e **recall** (detectar verdadeiros casos de saída).

Comparação de Desempenho e Seleção do Modelo: Os modelos foram treinados, ajustados e avaliados com base no F1-Score como métrica principal no conjunto de teste. Os resultados comparativos finais foram:

Tabela 6: Comparativo de desempenho entre os modelos.

Modelo	**F1-Score**
Regressão Logística	0.74
Random Forest	0.85
Gradient Boosting	0.87
XGBoost	0.87

Conforme os resultados apresentados, os modelos **Gradient Boosting** e **XGBoost** obtiveram os maiores F1-Scores, ambos atingindo **0.87**, superando os demais algoritmos testados. No entanto, apesar de o F1-Score ser igual, o **XGBoost apresentou um *recall* superior**, ou seja, conseguiu identificar uma proporção maior de funcionários que realmente saíram da empresa.

Essa diferença ocorre porque, ao configurar o XGBoost com o parâmetro `scale_pos_weight`, o modelo **abdicou parcialmente da precisão em favor de um maior recall** — uma decisão intencional, já que o objetivo do estudo era aumentar a capacidade de identificar os casos de saída (classe minoritária). Com isso, o F1-Score permaneceu estável, mas o modelo se tornou mais sensível e alinhado à finalidade prática da análise.

Considerando o equilíbrio entre desempenho e sensibilidade, o **XGBoost** se destacou como o modelo mais adequado para a previsão de *attrition*. Embora compartilhe o mesmo F1-Score do GBM, seu **recall mais alto** e o tratamento mais eficaz do desbalanceamento o tornam a melhor escolha para identificar colaboradores com maior probabilidade de deixar a empresa. Assim, o XGBoost combina **precisão suficiente**, **maior sensibilidade** e **robustez preditiva**, consolidando-se como o modelo final indicado para o projeto.